

Chrome DevTools Documentation

Project 2 — Giga Pet | COMP 484

Author: Kareem Ashouri

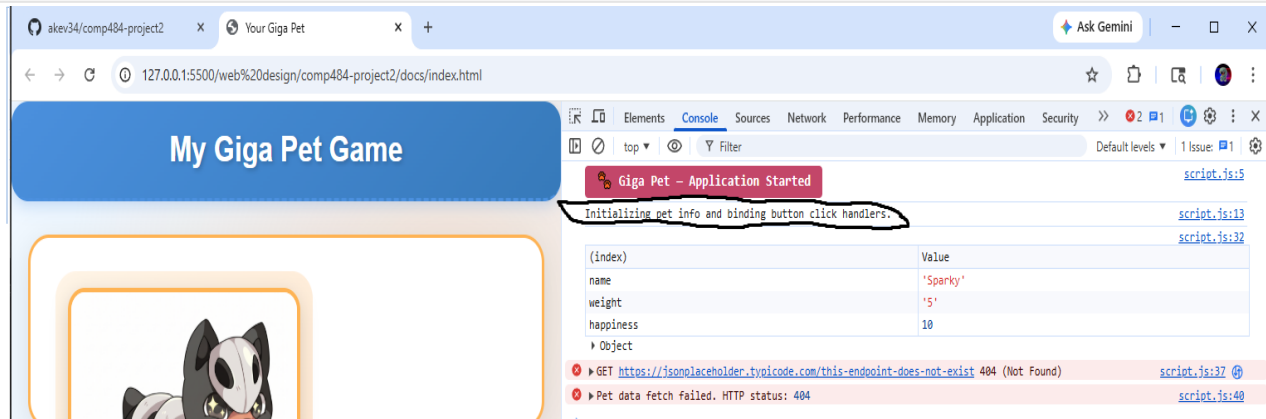
Overview. This document accompanies a modified copy of Project 2 (Giga Pet). The application has been instrumented with every console message type required by the assignment, three intentional browser-level errors, and one planted logic bug that is reproduced and fixed using Chrome DevTools breakpoints. Each section describes the implementation, points to the corresponding code in `Script.js`, and includes a placeholder where the captured screenshot should be inserted.

1. Message Logging

1.1 Log Info — console.info()

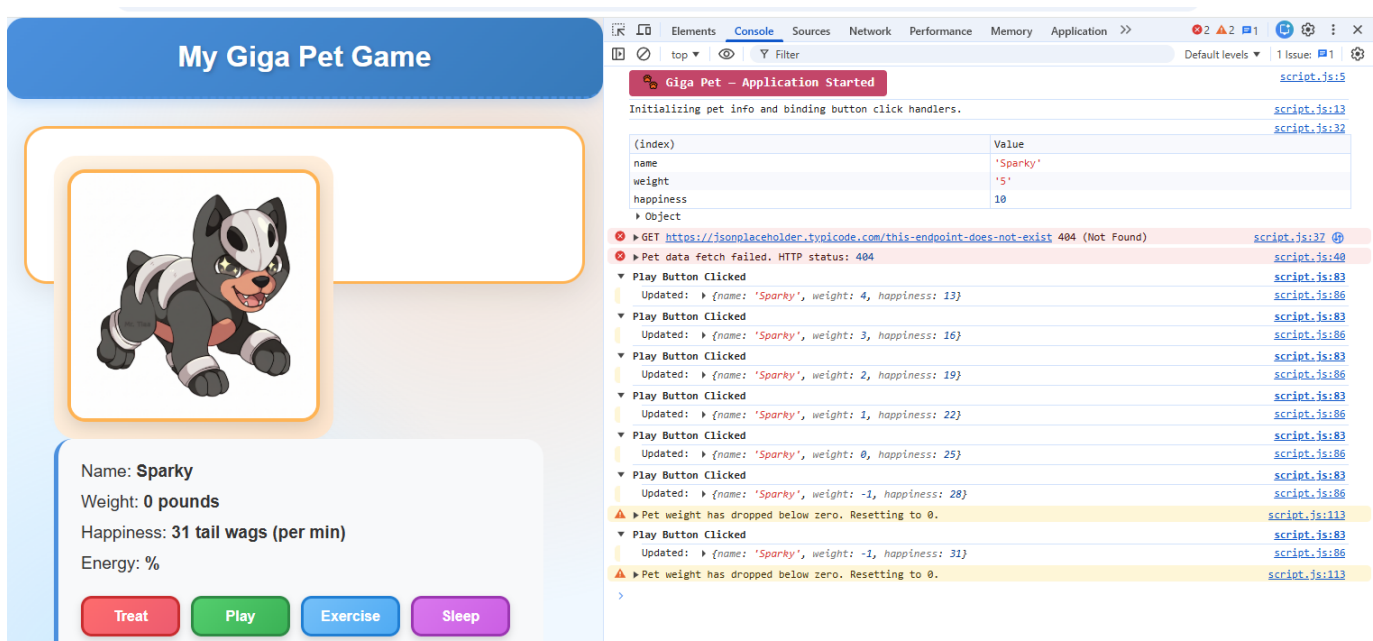
Marked as section **(B)** in the code. Fires once on page load to announce that initialization is starting. In Chrome, `console.info()` is rendered the same as `console.log()` but is filed under the **Info** log level for filtering purposes.

```
console.info("Initializing pet info and binding button click handlers.");
```



1.2 Log Warning — console.warn()

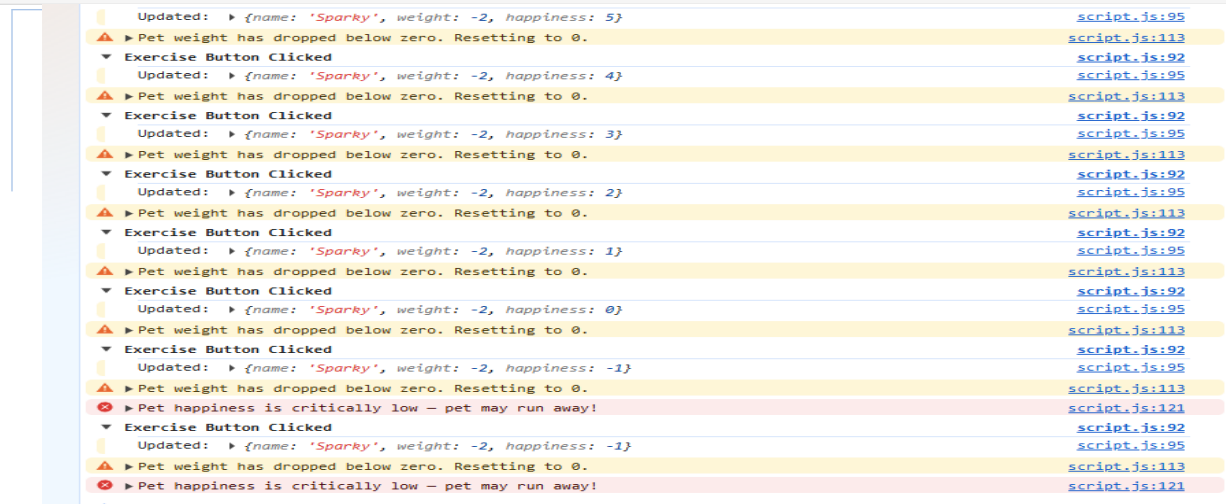
Marked as section **(F)**. Fires inside `checkWeightAndHappinessBeforeUpdating()` whenever the pet's weight drops below zero (a state reachable by clicking **Exercise** or **Play** repeatedly).



1.3 Log Error — console.error()

Marked as section **(G)**. Fires whenever happiness drops below zero. Renders as a red row with a stack trace in DevTools.

```
if (pet_info.happiness < 0) {  
  console.error("Pet happiness is critically low – pet may run away!");  
}
```



1.4 Log Table — console.table()

Marked as section **(C)**. Renders pet_info as a tabular grid showing index, key, and value columns — much more readable than a plain object dump.

```
console.table(pet_info);
```

Initializing pet info and binding button click handlers. script.js:13	
script.js:32	
(index)	Value
name	'Sparky'
weight	'5'
happiness	10
▶ Object	

1.5 Log Group — console.group() / groupEnd()

Marked as section **(E)**. Each button click is wrapped in a collapsible group that contains the before/after pet state. Useful for organizing related log output.

```
console.group("Treat Button Clicked");  
console.log("Before:", { ...pet_info });  
// ... mutation ...  
console.log("After:", { ...pet_info });  
console.groupEnd();
```

✖ ▶ GET <https://jsonplaceholder.typicode.com/this-endpoint-does-not-exist> 404 (Not Found) [script.js:37](#) ⓘ

✖ ▶ Pet data fetch failed. HTTP status: 404 [script.js:40](#)

▼ Treat Button Clicked [script.js:67](#)

Before: ▶ `{name: 'Sparky', weight: '5', happiness: 10}` [script.js:68](#)

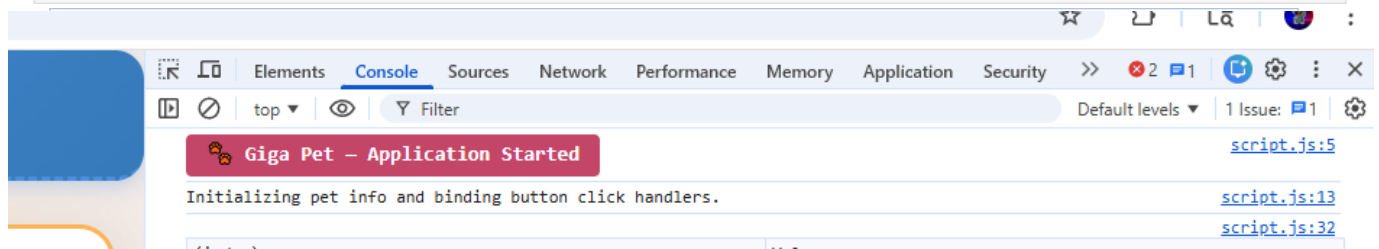
After: ▶ `{name: 'Sparky', weight: '51', happiness: 12}` [script.js:76](#)

↘

1.6 Log Custom — console.log() with CSS styling

Marked as section **(A)**. Uses the %c format specifier so the message is rendered with custom CSS — colored background, padding, and bold text — instead of the default monochrome console output.

```
console.log(
  "%c■ Giga Pet – Application Started",
  "color:#fff; background:#c44569; padding:6px 14px; border-radius:4px; font-size:14px; font-weight:bold;"
);
```

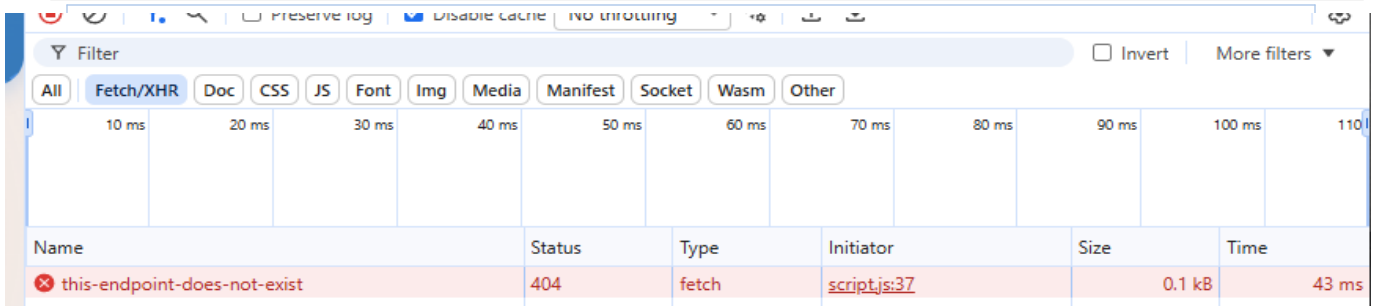


2. Browser-Logged Messages

2.1 Cause a 404 Network Error

Marked as section **(D)**. On page load, the app attempts to fetch a non-existent endpoint. Chrome logs the failed request to the Console (and the Network panel) automatically.

```
fetch("https://jsonplaceholder.typicode.com/this-endpoint-does-not-exist")
  .then(...)
  .catch(err => console.error("Network error:", err));
```



The screenshot shows the Chrome DevTools Network panel. The 'Fetch/XHR' filter is selected. A single request is visible: 'this-endpoint-does-not-exist' with a status of 404, type of fetch, and initiator of script.js:37. The table below summarizes the request details.

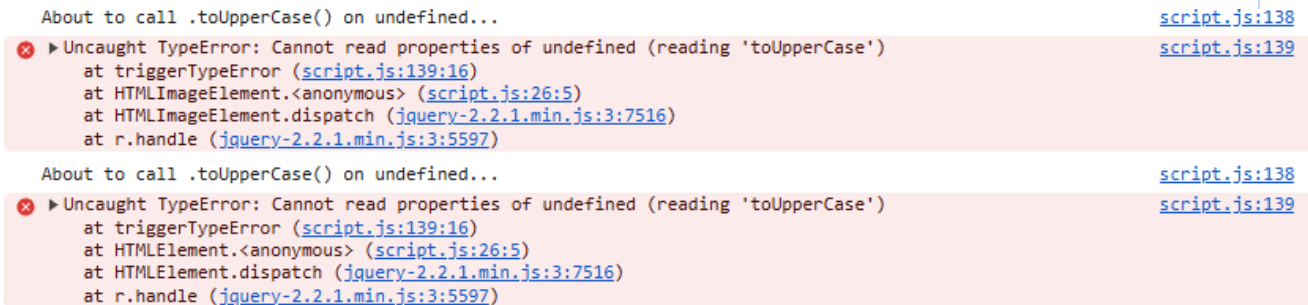
Name	Status	Type	Initiator	Size	Time
✖ this-endpoint-does-not-exist	404	fetch	script.js:37	0.1 kB	43 ms

```
✖ ▶ GET https://jsonplaceholder.typicode.com/this-endpoint-does-not-exist 404 (Not Found) script.js:37 ⓘ
✖ ▶ Pet data fetch failed. HTTP status: 404 script.js:40
> |
```

2.2 Cause a TypeError

Marked as section **(H)**. Clicking the pet image triggers `triggerTypeError()`, which calls `.toUpperCase()` on an undefined variable. Chrome logs the uncaught `TypeError` with a full stack trace.

```
function triggerTypeError() {
  var undefinedPet;
  undefinedPet.toUpperCase(); // throws TypeError
}
```



The screenshot shows the Chrome DevTools Console with a `TypeError` stack trace. The error message is 'Uncaught TypeError: Cannot read properties of undefined (reading 'toUpperCase')'. The stack trace includes the following frames:

- At `triggerTypeError` ([script.js:139:16](#))
- At `HTMLImageElement.<anonymous>` ([script.js:26:5](#))
- At `HTMLImageElement.dispatch` ([jquery-2.2.1.min.js:3:7516](#))
- At `r.handle` ([jquery-2.2.1.min.js:3:5597](#))

The console also shows the message 'About to call .toUpperCase() on undefined...' at [script.js:138](#).

2.3 Cause a Violation Warning

Marked as section **(I)**. The same pet-image click also schedules a `setTimeout` handler that busy-waits for ~250ms, exceeding Chrome's 50ms threshold and producing a *[Violation] 'setTimeout' handler took Nms* warning.

```
setTimeout(function () {  
  var start = Date.now();  
  while (Date.now() - start < 250) { /* block */ }  
}, 0);
```

[Violation] 'setTimeout' handler took 249ms

[script.js:147](#)

[Violation] 'setTimeout' handler took 249ms

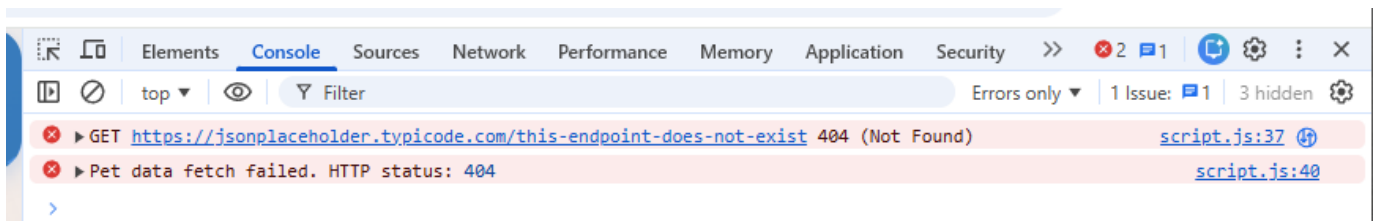
[script.js:147](#)

> |

3. Filtering Console Messages

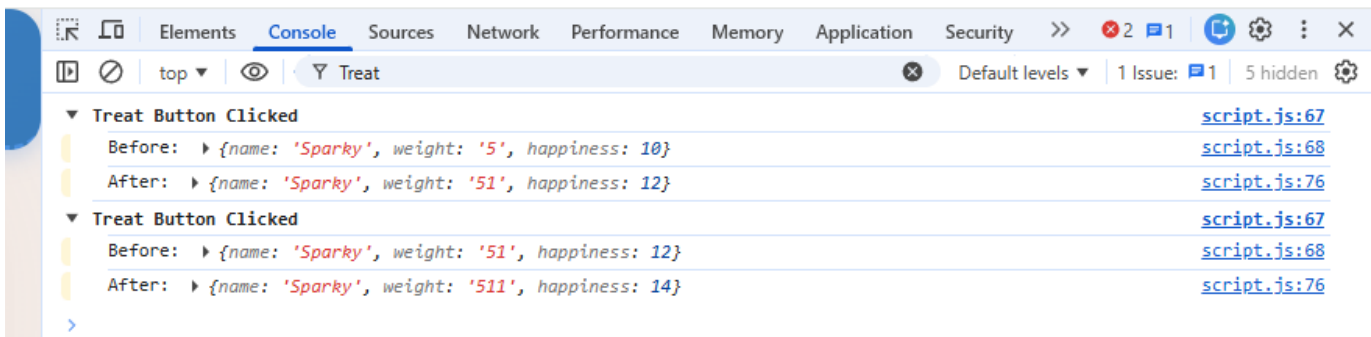
3.1 Filter by Log Level

The Console toolbar has a dropdown labelled **Default levels** (or **All levels**). Toggling these checkboxes shows or hides messages by category: Verbose, Info, Warnings, Errors.



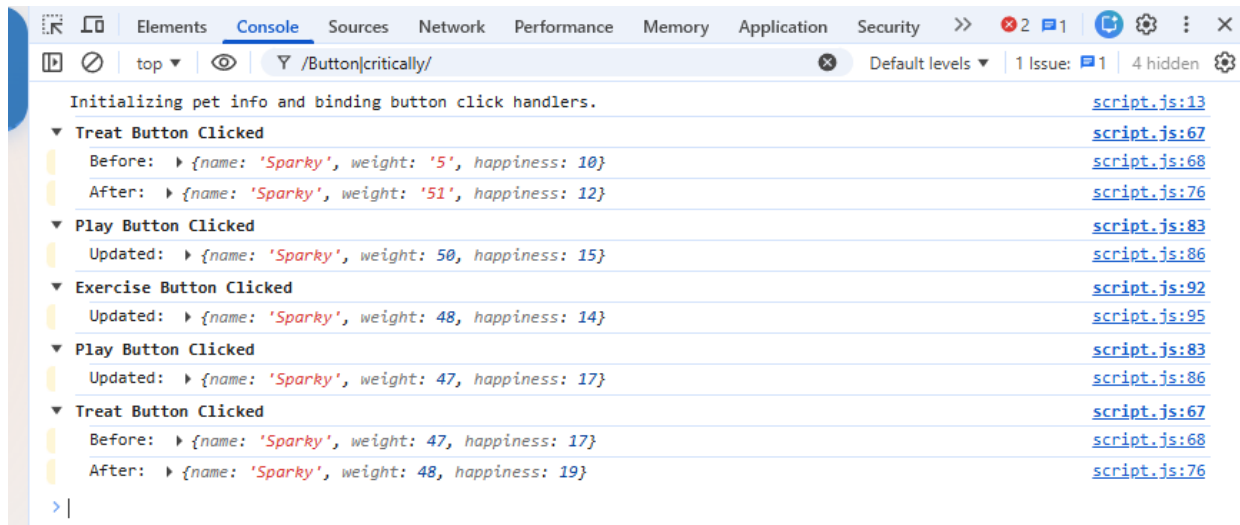
3.2 Filter by Text

The **Filter** textbox in the Console toolbar accepts a literal substring; only messages containing that substring remain.



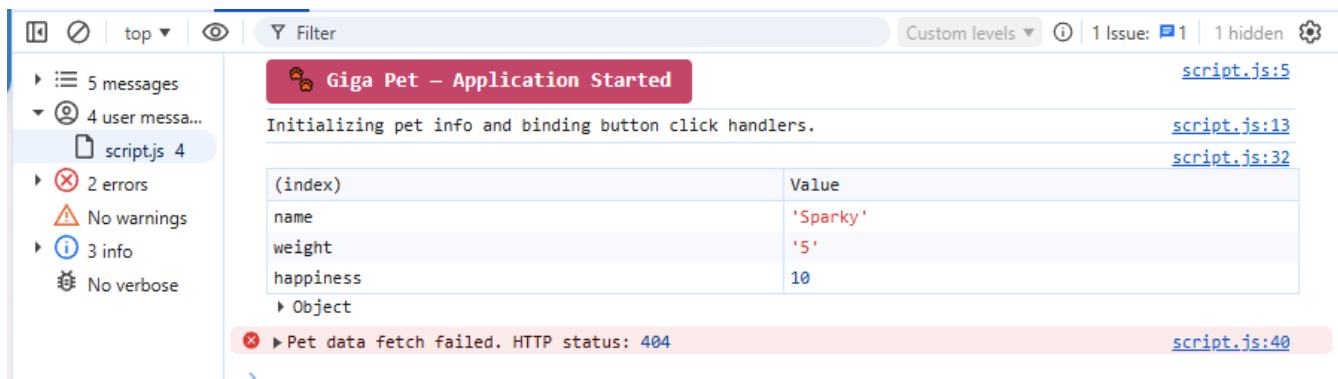
3.3 Filter by Regular Expression

Wrapping the filter input in forward slashes turns it into a JavaScript regular expression — useful for pattern matching across multiple message variations.



3.4 Filter by Message Source

The Console **Sidebar** (toggle via the side-panel icon in the toolbar) groups messages by the source file that produced them, letting you view messages only from `script.js` for example.



3.5 Filter by User Messages

The same Console sidebar contains a **User messages** entry. Clicking it shows only messages produced by my console.* calls and hides browser-generated messages such as the 404 and the Violation warning.

ElementsConsoleSourcesNetworkPerformanceMemoryApplicationSecurity>>

topFilterCustom levels1 Issue: 11 hidden

5 messages

4 user messages

script.js 4

2 errors

this-endpoint-does-not-exist 1

script.js 1

No warnings

3 info

No verbose

Giga Pet – Application Startedscript.js:5

Initializing pet info and binding button click handlers.script.js:13

(index)	Value
name	'Sparky'
weight	'5'
happiness	10

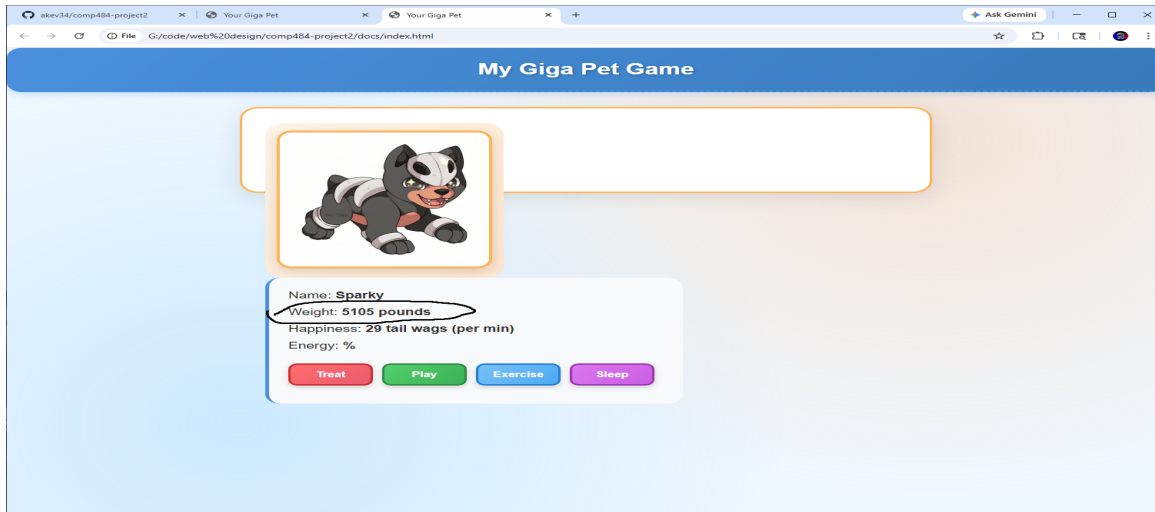
Pet data fetch failed. HTTP status: 404script.js:40

(index)	Value
name	'Sparky'
weight	'5'
happiness	10

4. Reproducing and Fixing a Bug

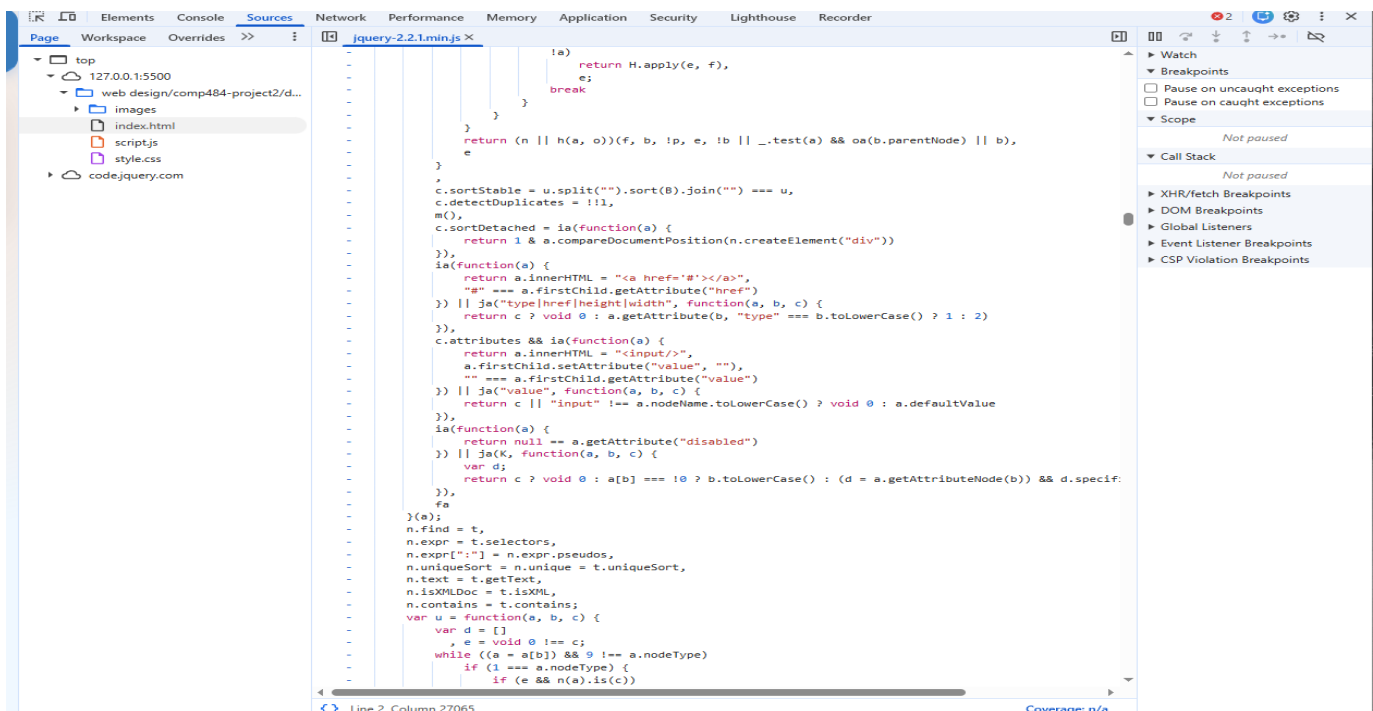
The planted bug. In `pet_info`, the `weight` property is stored as the string "5" rather than the number 5. The **Treat** button does `pet_info.weight = pet_info.weight + 1`, which JavaScript evaluates as string concatenation: `"5" + 1 === "51"`. Each subsequent click appends another 1, producing weights like 511, 5111, etc. This mirrors the canonical bug from the official Chrome DevTools debugging tutorial and is the bug we will investigate using breakpoints.

4.1 Reproduce the Bug



4.2 Get Familiar with the Sources Panel

DevTools' **Sources** panel has three regions: the file navigator (left), the code editor (center), and the debugger pane (right) which contains the breakpoint list, scope inspector, watch expressions, and call stack.



4.3 / 4.4 Pause the Code with a Line-of-Code Breakpoint

I set a line-of-code breakpoint on the buggy line — the assignment `pet_info.weight = pet_info.weight + 1;` inside `clickedTreatButton()`. When I click the Treat button, execution pauses at that line before the addition runs.

```
72 // breakpoint debugging walkthrough.
73 pet_info.weight = pet_info.weight + 1;
74 pet_info.happiness = pet_info.happiness + 2;
75
76 console.log("After:", JSON.parse(JSON.stringify(pet_info)));
```

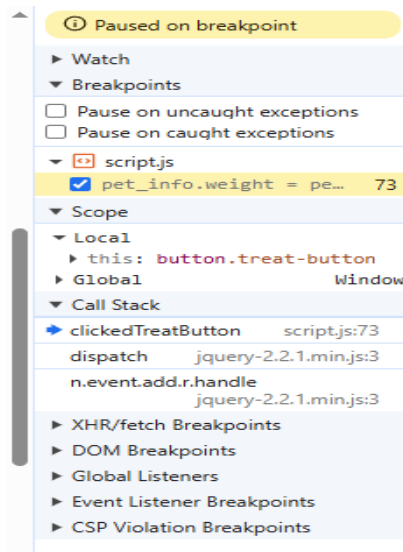
4.5 Check Variable Values

While paused, hovering over any variable in the editor reveals its current value in a popover — a fast way to inspect state without opening side panels.

```
51 // =====
52 // PET DATA
53 // NOTE: weight is intentionally stored as a STRING "5" (not the number 5).
54 // This is the planted bug used in the "Reproduce a bug" / breakpoint
55 // debugging walkthrough – it mirrors the classic Chrome DevTools tutorial
56 // where "5" + 1 evaluates to "51" instead of 6.
57 // =====
58 var pet_info = { name: "Sparky", weight: "5", happiness: 10 };
59
```

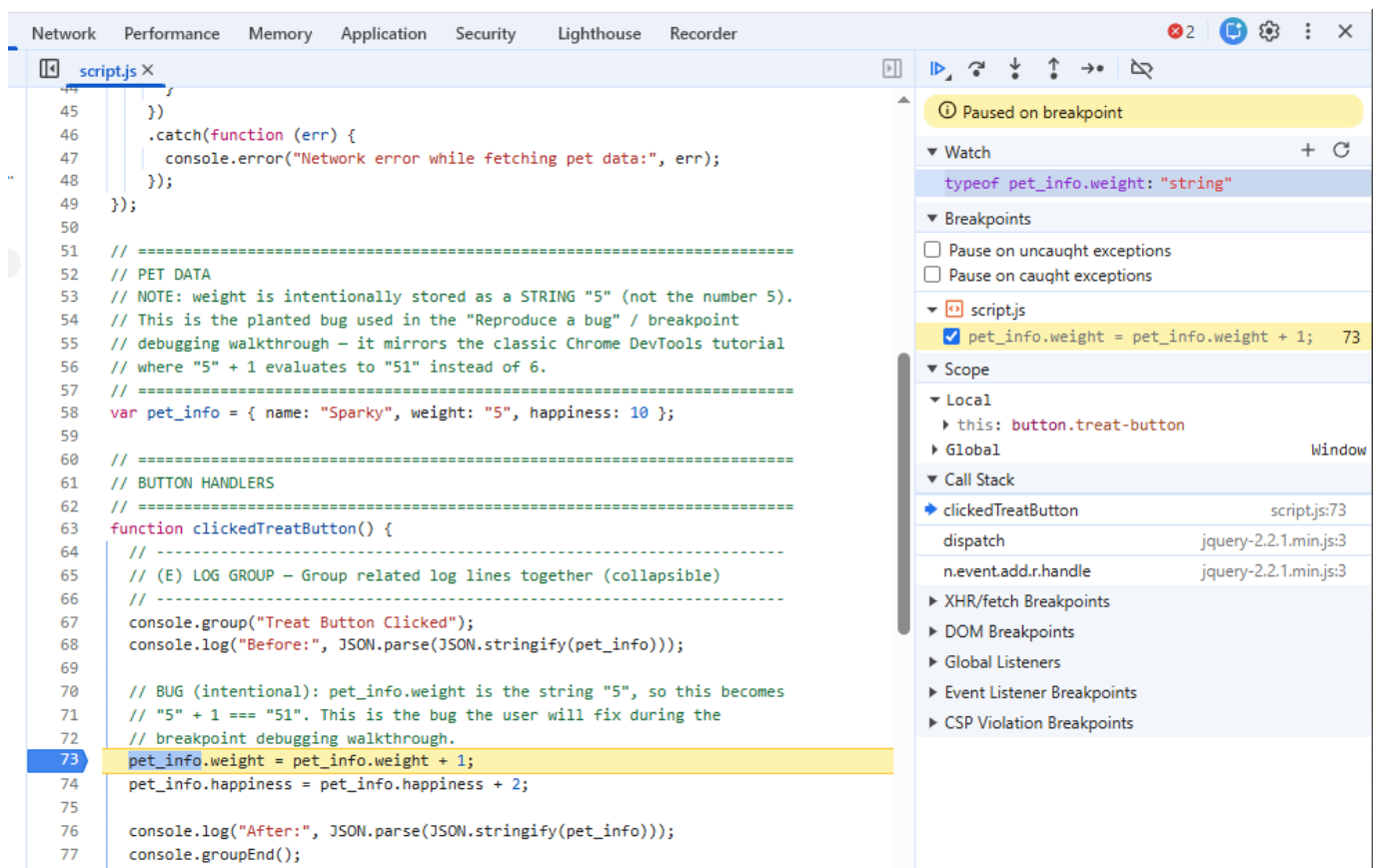
4.6 The Scope Pane

The **Scope** pane in the debugger sidebar shows every variable in the current scope (Local), the closure scope, and the Global scope. It updates as you step through code.



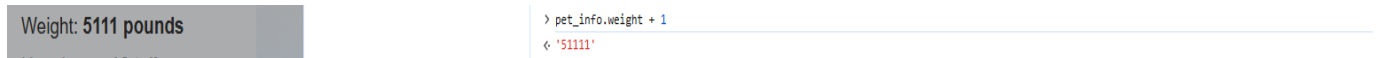
4.7 Watch Expressions

Watch expressions let you track the value of any expression as you step. To diagnose this bug I added a watch on `typeof pet_info.weight` — which will read "string", confirming the type problem.



4.8 The Console (While Paused)

While paused at a breakpoint, the Console runs in the scope of the current pause point. You can evaluate any expression involving local or global variables. This is invaluable for testing fixes before applying them.



4.9 Apply the Fix

Having confirmed the type issue, the fix is to coerce `pet_info.weight` to a number before adding. Either wrap it in `Number()` or initialize `weight` as a number in the first place. I applied both:

```
// 1) Initialize weight as a number  
var pet_info = { name: "Sparky", weight: 5, happiness: 10 };  
  
// 2) Defensive coercion in the handler  
pet_info.weight = Number(pet_info.weight) + 1;
```

▼ Treat Button Clicked	script.js:63
Before: ▶ {name: 'Sparky', weight: 7, happiness: 14}	script.js:64
After: ▶ {name: 'Sparky', weight: 8, happiness: 16}	script.js:69
▼ Treat Button Clicked	script.js:63
Before: ▶ {name: 'Sparky', weight: 8, happiness: 16}	script.js:64
After: ▶ {name: 'Sparky', weight: 9, happiness: 18}	script.js:69
▼ Treat Button Clicked	script.js:63
Before: ▶ {name: 'Sparky', weight: 9, happiness: 18}	script.js:64
After: ▶ {name: 'Sparky', weight: 10, happiness: 20}	script.js:69
>	

5. Summary

Requirement	Implementation Location
Log Info	script.js, section (B)
Log Warning	script.js, section (F)
Log Error	script.js, section (G)
Log Table	script.js, section (C)
Log Group	script.js, section (E)
Log Custom	script.js, section (A)
404 Network Error	script.js, section (D)
TypeError	script.js, section (H) — click pet image
Violation	script.js, section (I) — click pet image
Filter by Log Level	DevTools Console — log-level dropdown
Filter by Text	DevTools Console — Filter textbox
Filter by Regex	DevTools Console — /pattern/ in Filter
Filter by Message Source	DevTools Console — Sidebar
Filter by User Messages	DevTools Console — Sidebar
Reproduce a Bug	Section 4.1 — string concatenation bug
Sources Panel	Section 4.2
Line-of-Code Breakpoint	Section 4.3 / 4.4
Check Variable Values	Section 4.5 — hover popover
Scope Pane	Section 4.6
Watch Expressions	Section 4.7
Console While Paused	Section 4.8
Apply a Fix	Section 4.9 — Number() coercion

References.

- Chrome DevTools — Log messages: <https://developer.chrome.com/docs/devtools/console/log/>
- Chrome DevTools — Debug JavaScript: <https://developer.chrome.com/docs/devtools/javascript/>